CSCI 205 DESIGN MANUAL

ROCKY THE ROCKET DASHBOARD

SooAh Lay, Dipesh Bhattarai, Carlos Stolper, Jack McLaud

Technical Introduction

Our dashboard program, known as Rocky, provides an interactive and dynamic information dashboard that allows the user to get real-time measurements from sensors on a rocket unit and translate that into analytics that can be used to determine the stability, condition, location, and approximate landing site of the rocket unit. The internals of the dashboard are built upon via JavaFX and Java Scenebuilder. Rocky is designed to reduce overload for rocketry enthusiasts and professionals (future use) alike in the control room or out in the field, where they are provided with a single, cohesive interface that allows them to visualize and respond to metrics that matter to each state of the mission.

Prior to our actual development, we initialized our ideas by creating basic requirements that we wanted to see within Rocky:

1. Fluidity and Responsiveness: The GUI must run constantly at around 50-60fps regardless of how many data channels or modules are active at one given time
2. Design: The GUI must be distinct and separated into individual modules that each have specific use and their own toggle on/off
3. Expansion: The GUI must accommodate for changes in sensor on the actual rocket body
4. Testability and Reliability: Our model must pass a comprehensive array of tests that test its basic functionality and continuously launch and work as intended.

With the requirements we set, we went forth with the classic Model View Controller (MVC). This design pattern allows us to keep different logic and computational aspects separate. Our model section parses the incoming raw data logs as a .CSV file and converts them into visual altitude, gyro-stability data, barometric pressure, temperature, GPS, and telemetry information in real time. The model also includes real-time tracking of the current mission timeline: Pre Launch → Ascent → Apogee → Descent → Landed. The view is built upon JavaFX and Scene Builder components; altitude graphs and indicators, temperature graphs, a live map, and the status badge all share a common look but are separate modules that sit in their own self-containers, which allows for individual observation or a general analysis for the system. Our control section, mainly the DashboardController class, combines UI and model updates to stay in sync. As each layer is separate and has its own individual tasks, Rocky is agile and flexible for its tasks.

User Stories

The following user stories are ordered by sprints, denoted as (SX). Some are not labeled as user-stories within AIECode, however after reflection, we have determined them to be user stories and will attempt to change them once our product is finalized.

1. Mock-up User Interface (S1): This was a basic idea for what the UI would be, which aimed to capture a range of ideas we wanted to implement that the user would interact with. This became the foundation for what we would continue to develop.
2. Block Design (S1, trashed): The goal was to try and develop a skeleton template of the wanted GUI, with block outlines for specific relevant modules. Each module was 'blocked' in a specific section for easy placement later in the design.|
3. Block Design Revamped via Initial Design (S3): This was the actual implementation of the initial block design idea, combined with the initial UI design.
4. Button Activation (S3): This adds a visual cue and control so a user/operator can turn individual graphs on and off. This is based on our user persona's (Sara Harju) need to control specific graphs and see what sensors are currently active/not.
5. LiveFeed (S3): This adds a visual representation of the live-rocket feed. While we use a dummy video to represent the concept, this represents the capability of live telemetry that the user can view, which would be crucial and a need of all users.
6. Dynamic Graph for Gyro Data (S3): This allows the user to visualize the real-time gyroscopic data to see trends, behaviours, and or abnormalities that may require quick and decisive action.
7. Pressure and Temp. Visual Graph (S3): This also provides the user to visualize the real time barometric and temperature sensor readings as graphics, allowing users to monitor flight health and determine actions based on visuals and possible deviations.
8. Weather API (S3): This allows the user to see the current weather information in the local area (Lewisburg, PA), which provides information on launch-site conditions allowing changes based on weather forecast, projected wind and precipitation, and overall conditions.
9. Status of Rocket State (S4): This gives the user a visual capability of seeing what the current status of the rocket is, whether it is on the launch pad, mid-flight, landed, or possibly lost. This provides crucial information which affects real-time decision making regarding flight-health.
10. GPS Location of Rocket (S4): This will allow users to read real-time lat/long coordinates (translated onto a map) of a rocket, allowing for situational awareness and likely recovery attempts.
11. Velocity Graph on UI (S4): This allows the user to observe the specific velocity of a rocket in any possible state. The real-time updates provide information regarding the rocket's movement: increasing, decreasing, stalling, etc. This also provides crucial

information regarding the flight health and expected velocity values of a given rocket body.

12. Pressure and Temp (S4): This also provides crucial flight health information, primarily for the rocket body, ensuring that the fuselage is in operating condition and there are no overpressure and over temperature situations. If there are, then the information is relayed graphically and proper action can be taken.
13. Dynamic Graph for three (S4): This is a general change to the gyroscopic graph to ensure clean visualization, such as changes to the speed of the data presented and toggling of the graph: on/off.

Design

Rocky follows a relatively simple MVC+ model that separates the raw data processing, internal logic, visualization, and service processing. Our structure, as briefly reviewed in the introduction follows as:

1. Model: Telemetry information from sensors (Sensor.SensorData) and internal operations (SystemOperation) mesh to create a live and immutable view of the rocket's current state. The Sensor class holds a continuously added list of values (yaw, pitch, roll, temp., pressure, acceleration (x,y,z), etc) while the SystemOperation class tags each value for the proper mission stage, also determining the current state of sensors.
2. Controller: A combination of JavaFX controllers (DashboardController) that translates UI interaction into model updates.
3. View: Visual widgets and modules (LaunchGraph, LiveFeed, CountDownClock, PressurePlotApp, etc) that are bound and show observable properties. Each of the widgets and modules has their own FXML layout (including styling) to ensure that they do not disrupt the neighboring ones.
4. Services: Data-pipeline processors and helpers that get, filter, and format data. Their main role is processing information thus they are off to the sidelines.

CRC Cards

Below are our CRC cards:

Class: Rocket Dashboard	
Responsibility	Collaboration
• Display all dashboard components	• Sensor, Location, CountdownClock, SystemOperation, LaunchGraph, Timeline, Recovery, LiveFeed
• Update dashboard data in real-time	• Sensor, Location, CountdownClock
• Manage user interactions	• SystemOperation, CountdownClock

Class: Sensor	
Responsibility	Collaboration
• Collect pressure data • Collect temperature data • Collect accelerometer data • Collect gyro data	

Class: Location	
Responsibility	Collaboration
• Track current latitude/longitude • Update velocity • Update altitude • Display rocket trajectory on a map	

Class: CountDownClock	
Responsibility	Collaboration
• Display countdown timer	
• Update launch status	• SystemOperation
• Display live weather	

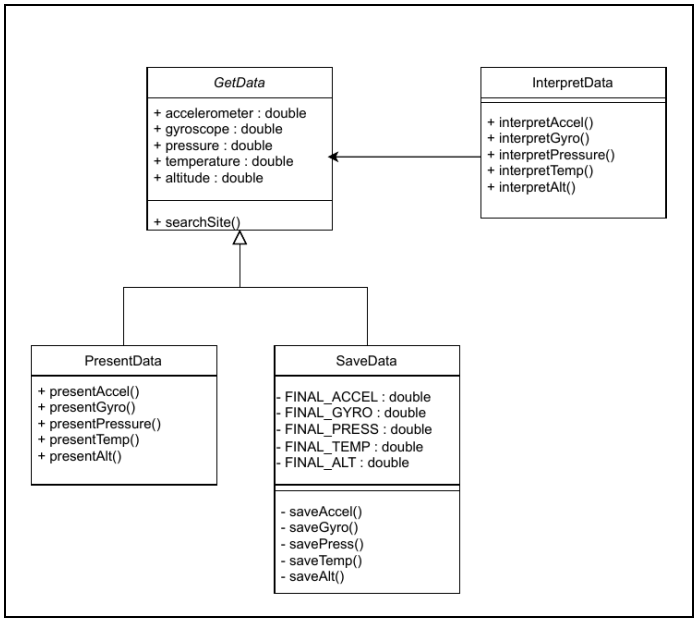
Class: SystemOperation	
Responsibility	Collaboration
• Manage power state (On/Off) • Display voltage • Control command operations (BM, MPU, TLM, GPS)	

Class: LaunchGraph	
Responsibility	Collaboration
• Display height vs. time graph	• Altitude

Class: Timeline	
Responsibility	Collaboration
• Store last seen latitude/longitude	• Location

Class: LiveFeed	
Responsibility	Collaboration
• Stream live video from launch station	

UML Diagrams



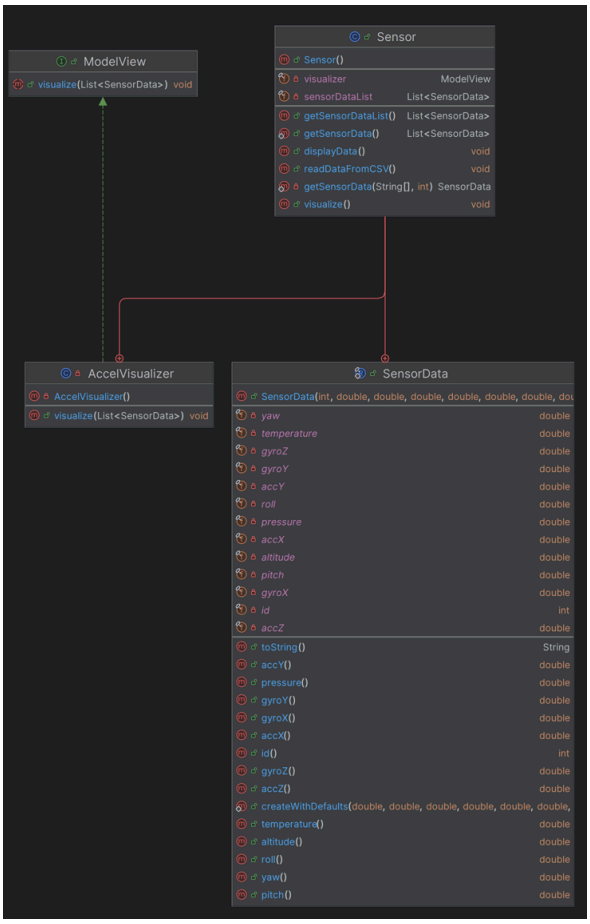
Our UML design features a service lane that processes data through a pipeline of modular components: GetData, InterpretData, PresentData, and SaveData. This design allows for flexibility, enabling us to easily integrate different data sources like real-time telemetry by replacing the .CSV reader without significantly altering the overall process.

The model lane (not shown) contains core data structures, including Sensor, Sensor.SensorData, and Location.

Acting as an intermediary, the controller lane (not shown) manages interactions between the user interface (e.g., button presses) and model events. It only communicates UI-related information and changes.

The view lane (not shown) comprises UI widgets such as LaunchGraph, LiveFeed, and CountDownClock. These widgets are linked to observable fields within the model, ensuring that UI elements dynamically update based on changes in the underlying data (for example, the LaunchGraph updating in response to altitude changes).

Screenshots and Images



To the right is a diagram from IntelliJ that shows Rocky keeps the data collection and visualization aspects separate. The Sensor class gathers each input parameter into a list of SensorData objects (e.g. yaw/pitch/roll, temperature, etc). The model also holds a reference (named visualizer) that points to any object implementing the ModelView interface, allowing us/users to add any graph without major modification.

Because our Sensor class only talks to the ModelView interface and not to specific graph classes, we can introduce new styles of graphs, say a new variable visualizer, without changing major sensor line code. The data collection aspect stays within the Sensor class (M); the visualization logic lives within the ModelView implementations (V), and the two interact via a method call, e.g. visualizer.visualize(sensorList) (C).

Interactions and Usage

Let's say that a user, a flight control analyst clicks the MPU6050 button. This would toggle the acceleration, gyroscopic, and yaw/pitch/roll graphs to stop plotting the information.

1. First, JavaFX would fire the actionEvent and the IDE (intellJ in this case) would inject the call into the DashboardController class → DashboardController.handleMPU6050Toggle(), which flips the boolean ‘isMPUon’, which then would change the MPU button text to “MPU6050 off” (C)
2. Inside the class, the controller would toggle the nine text labels beside the graphs to output “[GraphLabel]: OFF”. The toggling would also prevent the execution of updating the chart values, such that the loop inside is contingent on ‘isMPUon’ being True. (V)
3. The Sensor object just buffers the incoming SensorData; the data values are not rendered until ‘isMPUon’ is back to ‘True’. (M)
4. When the button is clicked and MPU6050 is enabled once again, DashboardController pushes ‘IsMPUon’ and removes the text labels status of being ‘OFF’ to showcasing the appropriate values and removes the buffering of the data.